

Verilog 第 5 次实验报告

学生姓名	王一诺	学号	2312331	指导老师	董前琨老师
------	-----	----	---------	------	-------

一、实验内容及步骤

参照循环语句中的例子，完成一个 32 位二进制数的乘法模块设计：

```
module multiplier_32bit (outcome, a, b);
    output reg [63:0] outcome; // 声明输出口，乘积结果为64位宽
    input [31:0] a, b; // 声明输入端口，两个乘数a和b各为32位宽
    integer i; // 声明一个整型变量，用于for循环中的迭代
    // always块，当输入a或b发生变化时触发
    always @(a or b)
    begin
        outcome = 0; // 初始化乘积结果为0
        for (i = 0; i < 32; i = i + 1) // 循环遍历乘数b的每一位
            if (b[i])
                outcome = outcome + (a << i); // 如果b的第i位为1，则将a左移i位后加到outcome上
    end
endmodule
```

测试文件：

```
module multiplier_32bit_tb();
    // 定义测试模块的信号
    reg [31:0] a; // 被乘数a
    reg [31:0] b; // 被乘数b
    wire [63:0] outcome; // 结果
    // 实例化32位乘法器模块
    multiplier_32bit uut (
        .outcome(outcome),
        .a(a),
        .b(b)
    );
    integer i;
    initial begin
        // 循环多次以进行多次测试
        for (i = 0; i < 10; i = i + 1) begin
            // 生成随机数a和b
            a = $random;
            b = $random;
            #1; // 等待一个时间单位，确保乘法器有足够的时间计算结果
            // 打印结果，便于观察和分析
            $display("Test %d: a = %h, b = %h, outcome = %h", i, a, b, outcome);
        end
        // 结束测试
        $finish;
    end
endmodule
```

测试样例：

Name	Value	
> a[31:0]	01110010101011111111011111100101	01110010101011
> b[31:0]	10111011110100100111001001110111	10111011110100
> outcome[63:0]	0101010000100100110100011011010101001101000010100011010101110011	01010100001001

图中所在时刻所示， $01110010101011111111011111100101 \times 10111011110100100111001001110111 = 0101010000100100110100011011010101001101000010100011010101110011$ 验证正确。

二、总结

1.编写功能清晰、结构化的代码有利于后续的调试和维护，要学会适当添加注释，大大提高代码的可读性，也有利于纠错改写。

2.在实践中遇到的问题往往能使人更深入地掌握知识和理解理论。比如这次编写时，对模块文件和 tb 文件进行测试都没问题，但一进行仿真就报错：**[USF-XSim-62] 'compile' step failed with error(s).**

怀疑是语法错误或者信号尺寸不匹配，最后发现是误把[32:1]认为和[31:0]一样，在模块定义中用[31:0]而在 tb 中用了[32:1]来定义信号位宽导致的。可见标准的定义声明是很重要的。